Phase I Projects

Your goal will be to develop an AI story generation system that uses LLMs in some way. Your generators should produce a crime mystery. More information on crime mystery stories is provided below.

You will select one of the project templates, which lay out some high-level guidelines to be met. It is expected that your story generator will use an LLM as a component and that other non-neural components will have be developed to interface and guide the LLM. For example some of the templates require a graph data structure representing the state of the story world or progression of events to be maintained, with the LLM proposing additions or details.

The templates are based off research literature on AI story generation, but the research literature we draw from does not consider crime mystery stories. Thus, even when templates are based heavily on specific research papers, there will be elements for which some novel innovation will be required to fit the technique to the assignment. The details of how you mesh the requirements of the template, the background literature, and the assignment requirements will be the crux of your initial team proposal, the first of several milestones.

Milestone 1: Proposal Video

In this milestone, you will create a presentation as a video recording that indicates the template you will use, and your rough sketch of how you will use LLM and non-LLM components, and how you think your proposed solution addresses the crime mystery requirements. The most straightforward approach will be a video made of slide presentations with voice-over.

Milestone 2: Cross-Team Proposal Assessments

You will review the proposal videos of other teams and provide constructive assessment of what elements from lecture might improve the proposal. Teams will not be required to act upon these and indeed will never see other teams' assessments. The goal of this is to reflect on what other teams are doing and to reflect on lecture material.

Milestone 3: Final Deliverables

There are two deliverables:

(a) A final presentation and demonstration **video**. Your final presentation will explain how your system works, and demonstrate your system in action. You are not

required to record your system running in real-time, but it should provide examples of the outputs of one or more successful runs and one or more unsuccessful runs.

(b) Your **codebase**, fully documented so we can read through the code to see how it works. The code should be runnable.

Milestone 4: Cross-Team Proposal Assessments

You will review the final videos of other teams and provide constructive assessment of what elements from lecture might have improved the final design. Teams will not be required to act upon these and indeed will never see other teams' assessments. The goal of this is to reflect on what other teams are doing and to reflect on lecture material.

Milestone 5: Self- and Team-Assessment

You will provide a brief document reporting on your role on the team project and your assessment of your performance as a team member. You will also report on the roles your teammates took and your assessment of their performances as team members. The purpose of this document is to help the instructional team determine whether the entire team should receive the same grade.

Grading rubrics for all deliverables will be provided separately

Theme: Crime Mystery

There are several core elements to a crime mystery. First, a crime is committed. It could be a theft, a murder, etc. Typically, the criminal is unknown, and a detective is tasked with figuring out who performed the crime. There are other characters that are suspects. A character becomes a suspect if they have the means (do they have the tools, physical capabilities, financial resources?), motive (do they have a reason to commit the crime?), and opportunity (could they have been at the scene of the crime at the time of the offense?).

The detective must eliminate suspects by determining if they did not have the means, did not have a motive, and did not have the opportunity. For example, an alibi is a statement of a suspect's whereabouts at the time of the crime, which could be a lie and must be verified. The story is the sequence of events that occur as the detective uncovers who committed the crime and how the crime was committed. Some of the things that make this story interesting:

- Multiple characters with means, motive, and opportunity.
- Characters with reasons to provide false alibis.

- Red Herrings: misleading clues or pieces of information that distract from the real issue.
- Clues that require multiple steps to obtain (e.g., physical obstacles to overcome; a character that must be convinced to give up information; etc.)
- A criminal who is actively working to hide, remove, or obfuscate clues.

There is a second story: the hidden story of what led up to the crime and how it was committed. This story is often told by the detective when they have all the information. Sometimes this story—or parts of the story—is told to the audience in advance; the details are not knowable by the detective at the beginning.

For the project templates below you might find that you need to create a system that generates two separate stories, the crime-solving story referencing details and facts produced by the crime-committing story.

Story Length

The crime mystery stories should consist of at least 15 plot points. A plot point is a consistent chunk of action that moves the plot forward. We distinguish between plot points and natural language content because a plot point could be expressed as a sentence, a paragraph, or a page, depending on the level of details. Thus, word counts can always be arbitrarily extended by generating more minute details. By measuring plot points, we are looking for at least 15 substantial things to happen in the story that change the state of the story world in non-trivial ways. For example, finding a clue, confronting a suspect, bribing a witness, etc.

The 15+ plot points should be related to the solving of the crime. You may have to generate a story about the crime and a separate story that solves the crime. The story about the crime has no length requirements.

Project Templates

1. Suspense Generation

Suspense is an affective response of readers/viewers to story content. One of the more thorough theories as to what causes suspense is as follows. It occurs when (a) the reader/viewer has affinity for a character—usually a protagonist—and thus cares about the fate of the character. It happens when (b) the reader/viewer believes that the protagonist has an important objective, and (c) their fate will be dire if that objective is not met. Events

happen that (d) make achieving their objective, and thus avoiding the dire fate, less easy to achieve and thus less likely. For example, a character we care about doesn't know there is a monster in their house with them. They are completely engaged in some task of great importance to them and want to complete it without interruption. Their friend knows about the monster and tries to call them to warn them, but gets bumped and drops their phone, which breaks. A way to avert their fate may or may not appear and the character may or may not avert their fate. Suspense is the process of slowly removing possibilities to avert their fate. Suspense is one way to raise the tension of a story. It is not related to surprise, because the reader/viewer knows of the danger and is frustrated that the protagonist does not. Surprise happens when the reader/viewer is suddenly presented with information they did not have.

LLMs are typically poor at generating suspense, tending to fall back on tropes. One way for an LLM to generate more suspenseful stories is to have an external process guide the generation process. This external process iteratively walks the LLM through each stage in creating suspenseful tension. For example, the LLM can be prompted to create a character, give the character a goal with important stakes, and then iteratively present the character with obstacles to success, making success have increasingly lower probability. Often a countdown mechanism is introduced so that the character is up against time and doesn't have infinite tries to succeed.

Implement the iterative prompting meta-controller technique in conjunction with the mystery. An iterative prompting meta-controller is an external control loop that walks the LLM through a process of creating all the raw materials for the story and then assembling those raw materials into a final story product. For example, for a spy thriller we would use an LLM to create a character like this:

> "Tell me about the name and the occupation of a protagonist of a spy thriller. Tell me about a very concrete goal the protagonist wants to achieve. Tell me about a dire situation that the protagonist would be put in, if the protagonist couldn't achieve the goal"

The materials for the plot points for a spy thriller would involve prompts like this:

> "Tell me about a concrete action the protagonist is most likely to take, in order to achieve the goal."

Then:

> "The protagonist tries to perform the following action: $action1. But the protagonist still fails to achieve the goal for a reason. Tell me about what this reason could be."

Note that this prompt refers back to the details returned by a previous prompt. Next:

> "The protagonist has taken all the following actions: [$action1]. But it turns out that those actions are not effective in helping the protagonist to achieve the goal, so the goal is still not achieved yet. Tell me about a concrete new action the protagonist is most likely to take next, in order to achieve the goal."

This process can be repeated several times. But no one wants to read the raw outputs of the LLM created like this, so a final phase would be to make a fluent retelling. Crime mysteries are going to involve more details about the crime and the actions of the protagonist to solve the crime.

Source material:

- Readers as problem-solvers in the experience of suspense, by Gerrig (see Canvas files for PDF)
- Creating Suspenseful Stories: Iterative Planning with Large Language Models https://arxiv.org/abs/2402.17119 (Note: this paper was done in the context of spy thrillers and would likely need to be adapted to crime mysteries)

Pitfalls:

- Suspense would happen in the uncovering of the crime, as opposed to the backstory of how the crime is committed. However, the two are inextricably linked.

Project Specific Questions:

- The original work on suspense was able to invent all details as the suspense story was generated. In the class project you may have details of the crime that may have been invented by a prior generation process. If you pre-generate the crime details, how do you adapt the technique to use the details? If you generate the crime details as you generate the suspenseful crime solving, how do you ensure that the crime details are consistent with each other (i.e. not a trivial crime to solve or crime details that are unconnected)? For your proposal, describe how you intend to address these potential issues. For your final demo video, describe how you finally solved them.

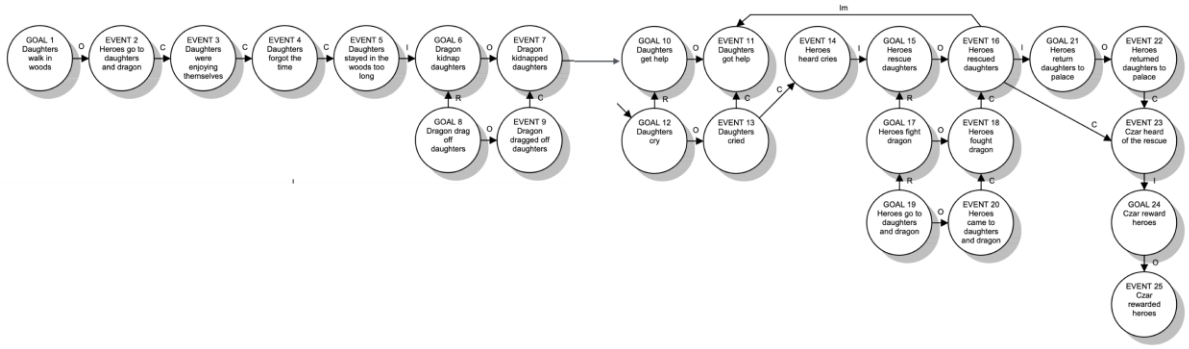
2. Reader Model Driven Story Generation

A reader model is a model of what the reader/viewer is expected to believe about the story world. The most well-developed theory of reader mental models is the QUEST model, which suggests that readers track the events of the story and goals of characters. Readers

also track the causal relationships between events (event B could not have happened unless event A happened first); relationships between super-ordinate and sub-ordinate goals (a character had the sub-ordinate goal to do A because it was necessary to reach super-ordinate goal G); and relationships between events and goals (event A caused a character to have goal G). The QUEST model tells us that readers understand a story because they can explain how events and goals are related to each other. That is, the reader can answer why or how questions about any part of the story. And if the there are parts of the story for which the model doesn't explain why or how, then the story is broken and/or incomplete.

The following is an example story, and the reader model that it produces:

Once there was a Czar who had three lovely daughters. One day the three daughters went walking in the woods. They were enjoying themselves so much that they forgot the time and stayed too long. A dragon kidnapped the three daughters. As they were being dragged off they cried for help. Three heroes heard the cries and set off to rescue the daughters. The heroes came and fought the dragon and rescued the maidens. Then the heroes returned the daughters to their palace. When the Czar heard of the rescue, he rewarded the heroes.



C: causal relations
I: goal initiation relations
R: sub-goal reason relations
O: goal outcome relations
Im: Implication (one thing is implied by another)

The normal use of QUEST would be to take a pre-written story and construct a QUEST graph and then analyze the graph for relations that the theory would tell us should be present but are not. To generate using reader models, we turn to engagement-reflection theory. Engagement-reflection (ER) theory tells us that writers alternate between two modes of content creation.

- Engagement: Generate events that come naturally with little regard for logical coherence.
- Reflection: Return to generated content to repair incoherence issues.

ER theory tells us that stories are thus not written linearly, but involves periods of linear content creation and periods in which the writer returns to existing content and adds new content in earlier parts of the story.

Engagement could be autoregression using an LLM. Reflection would then be a process of constructing a QUEST graph, identifying missing relations (no causal relation explaining how an event was able to happen; no relation explaining why a character performs an action), and introducing new content into the story.

Source Materials:

- Question Answering in the Context of Stories, by Graesser et al. (see Canvas files)
- MEXICA: a computer model of a cognitive account of creative, by Perez y Perez and Sharples (see Canvas files)

Pitfalls:

- LLMs like to get into minutia and dialogue. It should be prompted or tuned to generate at the appropriate level of plot abstraction.
- Infinite recursion: some events are commonsense enough that they do not need preceding causal relations to explain how they happen. The reflection process could recurse infinitely trying to explain everything and some means of deciding when to stop would be necessary.

Project Specific Questions:

- The default assumption of generation with LLMs is that the generation should be autoregressive in a forward manner. That is, each token is concatenated to the end of the context and the new context is used to generate the next token, and so on. This locks details in early, regardless of whether they are unnecessary or make forward progression of the story more challenging. How will you use the ability to generate in non-chronological order to take advantage of the crime mystery format? For your proposal, describe how you intend to address this question. For your final demo video, describe how you addressed them.

3. Story Planning with LLMs

Classical symbolic approaches to story generation would use state-action-space search planning. Assuming a story is a plan of actions for character, a state-action-space planner can guarantee logical coherence. Logical coherence means that each character action is possible because the preconditions for that action are met by preceding actions. For

example, if a character wants to get money out of a safe, they have to have the key. If there is not an action in which the character obtains the key (or if the character obtains the key afterward), then the story must be repaired by adding new actions to the story or by reordering actions. As each action added to the plan/story, the planner can verify that all the logical preconditions for the action are met, or revise the story plan to make sure that each precondition is met. One limitation of classical symbolic planning approaches to story generation is that they require a pre-specified library of known action templates. This limits the space of possible stories to action templates that can be identified in advance. One way to overcome this is to not require pre-known action templates in advance but to figure out preconditions and effects on the fly as they are needed.

A plan is a data structure consisting of:

- A set of actions. Each action has a set of preconditions and effects.
- A set of flaws: A flaw is a reason why the plan is not complete (see below).
- A set of temporal relations: Each temporal relation (a_1, a_2) indicates that a_1 must occur before a_2.
- A set of causal relations: Each causal relation (a_1, c, a_2) indicates that the precondition c of action a_2 is satisfied by a matching effect of action a_1. A causal relation indicates that between the time a_1 ends and a_2 begins c is true and must remain true. There must be a corresponding temporal relation (a_1, a_2).

There are two types of plan flaws, each of which has several strategies for repair:

- Open precondition flaw: some action a_i in plan p has a precondition c, that is not satisfied by the effect of another action guaranteed to occur before a_i.
    - Repair strategies:
        - Identify an existing action a_j that has an effect e that matches c and create a temporal link (a_j, a_i) and a causal link (a_j, c, a_i).
        - Create a new action a_k that has an effect e that matches e, create a temporal link (a_k, a_i) and a causal link (a_k, c, a_i).
- Causal threat: There is a causal relation (a_1, c, a_2) and another action a_3 has an effect e that is the negation of c, and a_2 may occur during the interval span indicated by the causal relation (between a_1 and a_2).
    - Repair strategies
        - Force a_3 to occur before a_1 with a temporal relation (a_3, a_1).
        - Force a_3 to occur after a_2 with a temporal relation (a_2, a_3).

With all the data structures in place, the planning algorithm is a straightforward search process that takes inspiration from A*:

1. Let G be a set of goal propositions $g_1 \dots g_n$
2. Let p_{init} be an empty plan, with open precondition flaws corresponding to each goal proposition.
3. Let Q be a queue initially only containing p.
4. While Q is not empty
    1. Let p_{cur} be a plan popped from Q, usually the one with fewest flaws.
    2. If p_{cur} has no flaws then return p_{cur}, the planner has succeeded
    3. Let f be a flaw in p_{cur}
    4. For each flaw repair strategy for the flaw type of f do:
        1. Let p_{new} be a clone of p_{cur}
        2. Apply the flaw repair strategy to p_{new}
        3. Remove flaw f from p_{new}
        4. Add any new open precondition and causal threat flaws to p_{new}
        5. Add p_{new} to Q

Actions are created when there are other actions that don't have preconditions satisfied. To create a new action, use an LLM to create an action with the required effect to satisfy the unsatisfied precondition. The action creation process would also create the preconditions for the new action. The effects can also be created at this time, or the open precondition flaw repair process could try to identify actions that should have matching effects at the time.

Source Materials:

- Neural Story Planning https://arxiv.org/abs/2212.08718

Pitfalls:

- Infinite recursion: Normally there would be an initial state that can satisfy preconditions, and eventually all actions will match to conditions in the initial state so that no further actions need to be generated. Since we might not have an initial state, some other means of deciding that further action generation is not necessary is needed (such as a notion that an action's preconditions can be taken as a "given" or "obvious").
- If preconditions and effects are sentences or phrases, then preconditions and effects that are semantically equivalent may not be recognized as matching.
- If there are many actions that can be created to satisfy a precondition for an action, then the branching factor of the planner can be very high. A means of limiting the branching factor might be needed, or a depth-first search strategy might be

applicable, thus only increasing the breadth as necessary when depth search needs to backtrack.

Project Specific Questions:

- Causal plannings techniques such as described here generate backwards from a goal, which ensures every element in the plan is causally sufficient and necessary. While it might seem straightforward to have a goal such as "crime is solved", the crime has many elements and facts. How can the planning algorithm above be adapted to factor in the various elements of the crime in terms of goal as well as when actions/events are added to the story plan? For your proposal, describe how you intend to address this question. For your final demo video, describe how you addressed them.

Planners can be a bit tricky to implement. See the reference code for an example.

4. Hierarchical Task Network Generation

A lot of practical planning applications use hierarchical abstractions. A hierarchical task network (HTN) is a means of creating plans starting with a task. A task is solved by a task instantiation, which is a knowledge structure contains (a) a sequence of sub-tasks, and (b) applicability constraints. For example, the task of traveling to another city might refer to one of several possible task instantiations:

A. Travel by car, consisting of sub-tasks:
    i. Go to car
    ii. Get in car
    iii. Drive to destination
    iv. Get out of car at destination

    And having applicability constraint:
    a. Must have a car
    b. Destination must be less than 500 miles away

B. Travel by plane, consisting of sub-tasks:
    i. Buy ticket
    ii. Go to nearest airport
    iii. Board plane
    iv. Fly to airport nearest to destination
    v. Deboard plane
    vi. Go to final destination

With applicability constraints:

a. Must have more than $500
b. Destination must be more than 500 miles away

C. Travel by train, with sub-tasks:

i. Buy ticket
ii. Go to nearest train station
iii. Board train
iv. Ride train to nearest train station to destination
v. Go to final destination

With applicability constraints:

a. Must have more than $100

Note the recursive structure, each sub-task may have several possible task instantiations with several sub-tasks of its own. Travel by plane might have a sub-task of travel to airport, which in turn might be solved by task instantiations of traveling by car or train. The result is a hierarchically composed network of task decompositions.

For storytelling, tasks will be plot points (or scenes) and task instantiations will be common ways that those plot points (or scenes) can be resolved. For example, in a soap opera, there may be a plot point in which a character blackmails another character. There may be several formulaic ways in which this sub-plot unfolds. The advantage of hierarchical task planning is that the sequence of sub-tasks does not need to be based on logical correctness but can include logically unnecessary sub-tasks for the purposes of making things more dramatic or interesting to the reader. For example, a blackmailer need not directly confront their victim, but that may make the blackmail more interesting.

The classical advantage of HTN planning is that it is fast. But this comes at the disadvantage of having a library of hand-crafted pre-scripted task instantiations. With LLMs, we can circumvent the library pre-specification stage by asking an LLM to generate one or more task instantiations and reason about their applicability constraints. An external planning loop can select a task instantiation and recursively decompose its sub-tasks.

Source Materials:

- Story-Telling as Planning and Learning, by Lebowitz (see Canvas files) (note: this implements soap operas instead of crime mysteries)
- ChatHTN: Interleaving Approximate (LLM) and Symbolic HTN Planning https://raw.githubusercontent.com/mlresearch/v288/main/assets/munoz-avila25a/munoz-avila25a.pdf

Pitfalls:

- Some task specifications must be primitive in the sense that they don't have sub-tasks. LLMs could decompose tasks without end unless there is a way to recognize when a task instantiation is primitive.
- There must be some way of representing state and recognizing the applicability conditions of task instantiations so that non-sensical task instantiations are not chosen.
- If there is a high-level task of "solve crime" then the task decompositions need to be sensitive to the exact nature of the details and facts created by the crime.

Project Specific Questions:

- Hierarchical plannings techniques such as described typically start with one over-arching goal and task such as "crime is solved", the crime has many elements and facts. How can the planner be made to factor in the various elements of the crime? For your proposal, describe how you intend to address this question. For your final demo video, describe how you addressed them.

Video Creation

The most straightforward means of creating a video will be a slideshow with voice-over, using screen video and audio recording.

Proposal videos:

- Introduce the team and team name.
- Indicate the template
- Describe what kind of crime stories and how you plan to make the crimes and the solving of the crimes non-trivial (clues, red herrings, suspects, etc.)
- Present the proposed system overview and/or system architecture
- Address potential pitfalls
- Address how you plan to address the template-specific questions

Final videos:

- Introduce the team and team name.
- Indicate the template. If your system has a name, give that as well.

- Describe what kind of crime stories and how you plan to make the crimes and the solving of the crimes non-trivial (clues, red herrings, suspects, etc.)
- Present the final system overview and/or system architecture
- Address pitfalls you encountered and how you addressed them
- Address the template-specific questions
- Provide the actual text output of some generated stories. Include stories that were "successful" and stories that were "unsuccessful". Explain what causes unsuccessful stories.

Videos do not have a required length. They should be as long as necessary to address the above points.

Architecture Diagrams

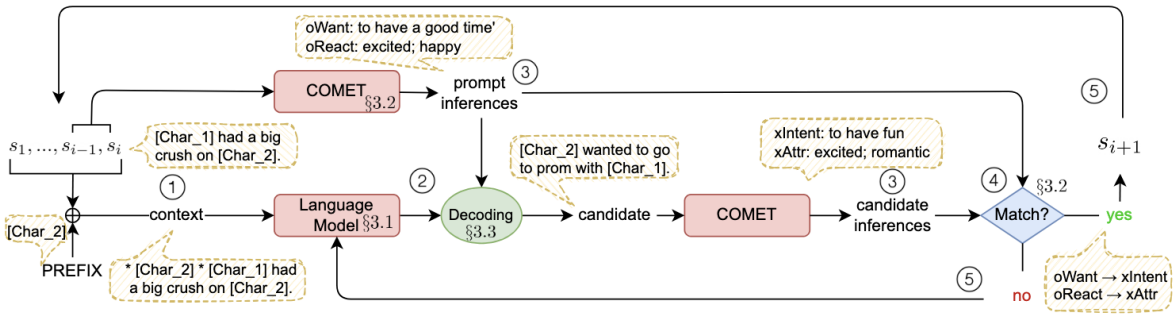
Architecture Diagrams are useful in helping people understand the components that are part of a complex system and how data flows through the system. There isn't a right or wrong way to do architecture diagrams.

As noted in the syllabus, generative AI should not be used to create visuals for your presentations. PowerPoint and other slide presentation software have good diagram drawing capabilities. Draw.io, built into Google Drive, is also very good.

We provide some examples below. We give the source papers for the diagrams, but you are not required to read or understand the papers. You are not required to emulate these diagrams. It can take some trial and error to figure out the right way to convey what your system does.

Example #1:

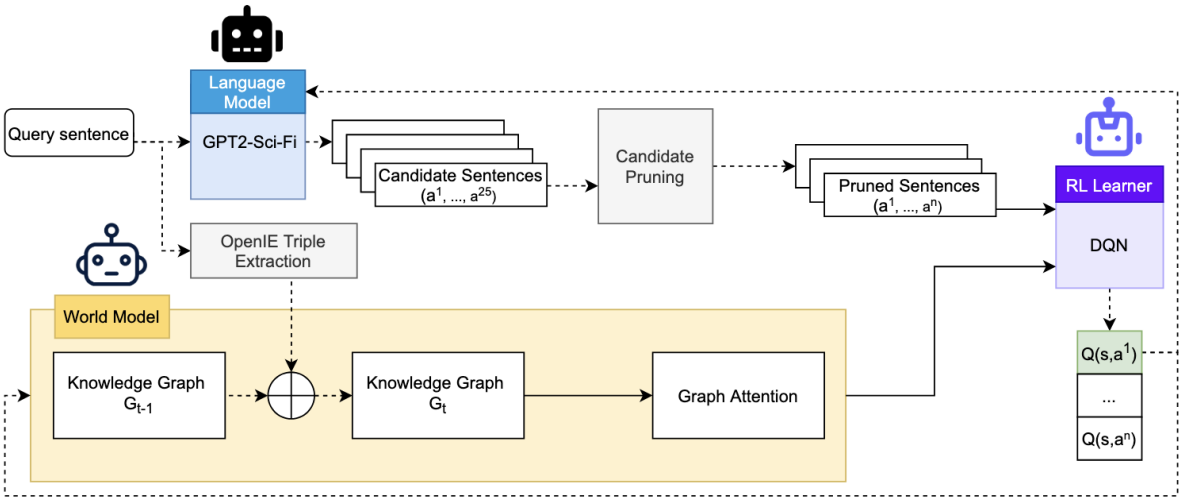
This diagram is for a story generation system that uses different language models doing different things at different stages of processing (COMET is a language model that takes a sentence and generates a set of candidate commonsense sentences that explain the input sentence). This example diagram uses callouts to give examples of what kind of information is passed between modules.



Source: https://arxiv.org/abs/2105.01311

Example #2:

This system is also a storytelling system that uses a pipeline approach. As in example #1, there are several AI systems operating in series, and the information being passed is made explicit. This system operates on graphs, even though the graphs are just expressed as boxes.

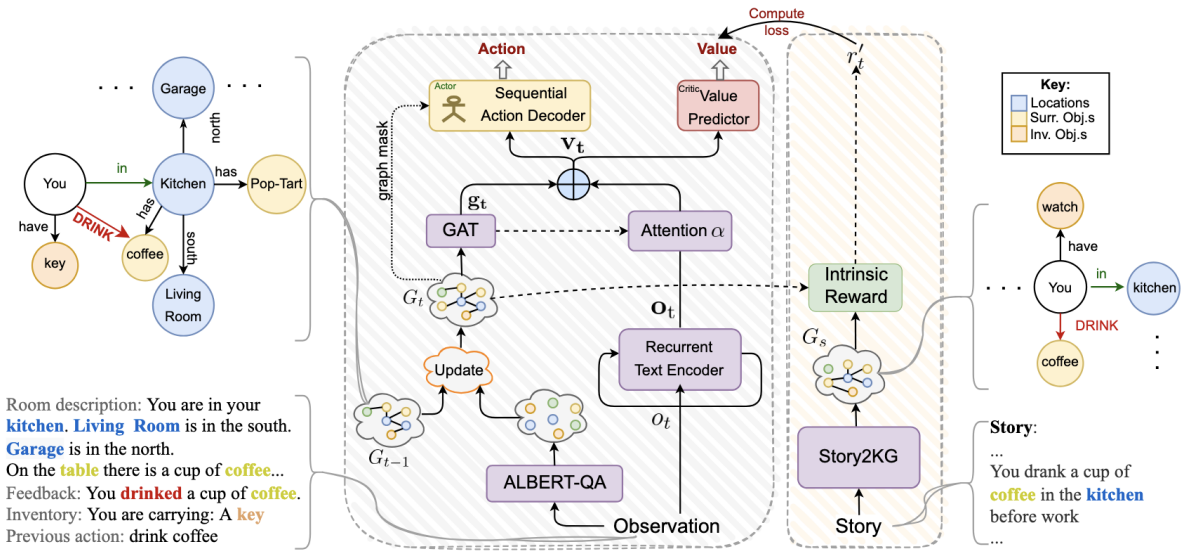


Source: https://arxiv.org/abs/2112.08593

Example #3:

This example is a bit more complicated. It also operates on graphs (there are two graphs, left and right, that are compared. A text description, bottom left, is converted into a graph by a process involving an LLM (ALBERT-QA is a version of BERT), and subsequently fed into other neural components (e.g., GAT is graph attention). Thus, you are seeing components that are more like layers in a neural architecture with graph information being fed in and extracted out. It is more likely that you will have completely self-contained AI/LLM modules

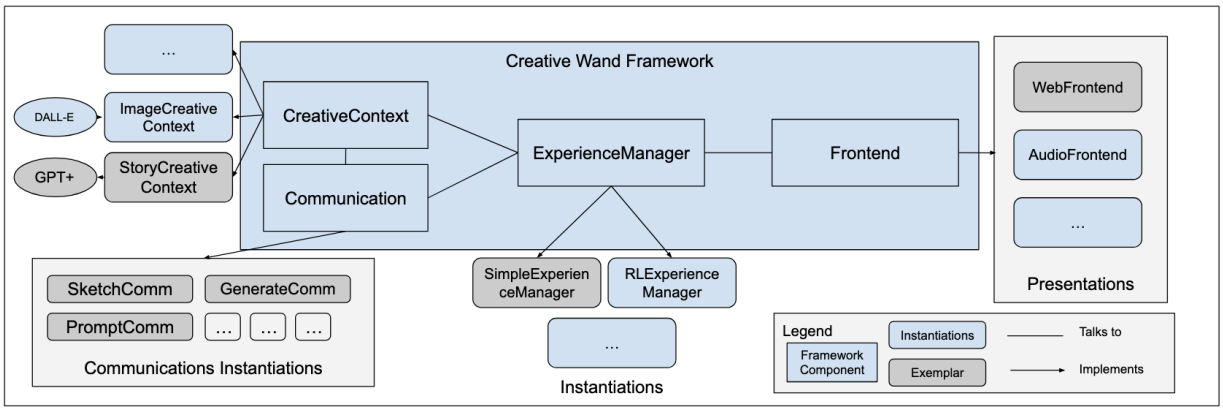
instead of needing to call layers out. We provide this as an example because it more explicitly shows what the graph data looks like.



Source: https://arxiv.org/abs/2301.10107

Example #4:

This is another pipeline example, with nested sub-components. It is for an authoring assistance tool, and the AI components are feeding information into a bunch of post-processing modules.



Source: https://arxiv.org/abs/2208.02886

Presenting with architecture diagrams

In a video presentation, it can be a good idea to walk through an example of how an input is processed by the system. You may have the entire architecture on the screen and verbally walk through how it works. Accompanying the walk-through with visual effects to focus the viewer's attention can be helpful. For example: highlighting various components as they are discussed; making components appear only when they are first introduced; or making example information being passed between components appear in the order that the modules are discussed.

Generated Stories in Final Videos

Your final video should have at least one story generated by your system that you consider to be a successful generation, and at least one story generated by your system that you consider to be an unsuccessful generation. For the latter, the run should be complete in the sense that the lack of success is not because of a system crash. Instead, an unsuccessful run is one where the system executes to completion but produces undesirable or unsatisfactory results. For example, a crime is solved in the story without uncovering all the necessary clues, or a story was not suspenseful when it should have been successful, or there are logical coherence or consistency issues.

Unsuccessful stories are sometimes referred to as "mis-spun" tales because of a paper about a story generation system that devoted a section to failed runs. Failed runs are instructive in terms of understanding the challenges of story generation and where one's system is prone to weaknesses. The final grade for the overall project will not be determined based on the mis-spun stories, or the severity of the lack of success in particular runs. The purpose of this is to provide the instructional team an understanding of the degree to which you were reflective of your system. All systems are expected to have varying degrees of success from run to run.

Cross-Team Assessments

In cross-team assessments, your team will review the videos from three other teams (which ones will be assigned). For each other team assigned to review, you will:

- Summarize their system

- Describe any pitfalls they may encounter that they do not already address in their videos (proposal stage), or summarize pitfalls that they encountered and how they addressed them (final stage).
- Describe how they intend or did address the template-specific questions (proposal stage), or how they did address template-specific questions (final stage)
- Suggest <u>two</u> things from lecture that would improve their lecture (proposal stage), or would have hypothetically improved their final design (final stage)

Note: cross-team assessments are not used in the grading of teams' proposals or final deliverables. It is used only to assess the reporting team's ability to connect the projects to lecture materials.

Cross team assessments should be 500+ words per reviewed system, for a total of 1500+ words.